

ML in Applications

***Dipartimento di Automatica e Informatica
Politecnico di Torino, Torino, ITALY***

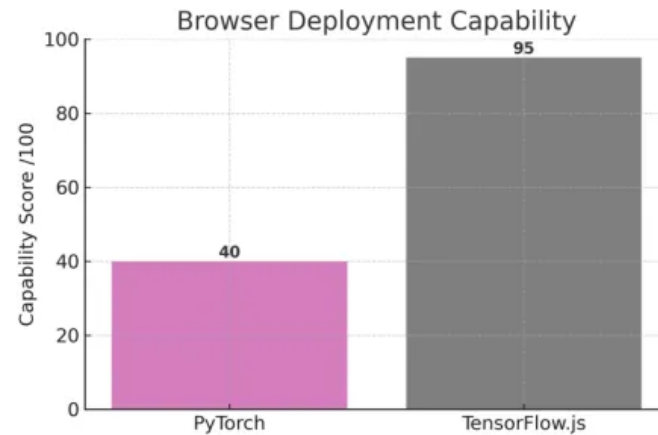
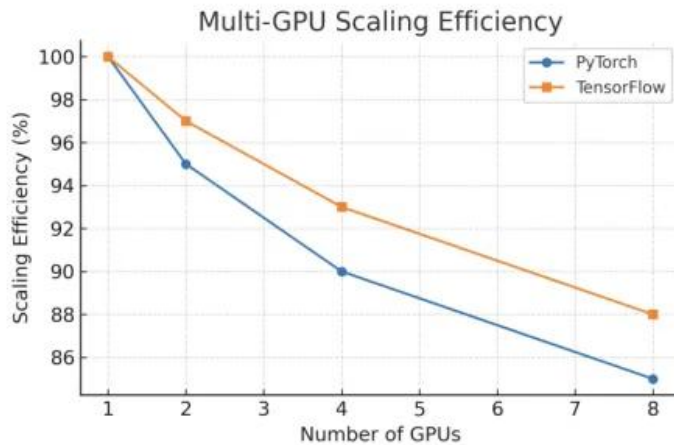
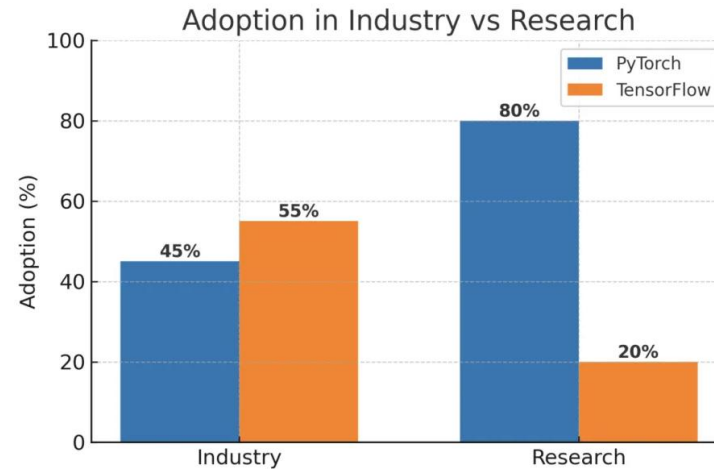
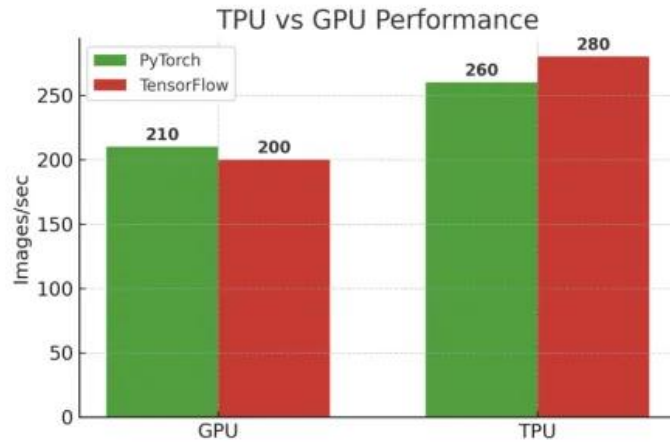


**Politecnico
di Torino**

Lab 4

Tensorflow - Introduction

Why Tensorflow?



source: <https://arxiv.org/html/2508.04035v1>

Why Tensorflow?

- **TensorFlow**
 - Stronger in production and deployment
 - Rich ecosystem (TensorFlow Lite, Serving, JS)
 - Better support for scalability (mobile, edge, cloud)
- **PyTorch**
 - More popular in research and academia
 - Flexible and intuitive (Python-first, dynamic graphs)
 - Preferred for rapid prototyping

Job Market & Career Impact

CAREERS

Framework demand in AI job postings and salary expectations

PyTorch

37.7%

OF AI JOB POSTINGS

SALARY RANGE

\$103K – \$207K

PyTorch-focused roles

TensorFlow

32.9%

OF AI JOB POSTINGS

SALARY RANGE

\$98K – \$173K

Avg: \$122,738 (ZipRecruiter)

28%

ML JOB GROWTH Q1 2025

Senior AI specialists earn \$200K–\$312K, with NLP and computer vision commanding the highest premiums. Python, TensorFlow, and PyTorch remain the top 3 most requested tools in ML job listings.



Be "Bilingual"

Most valuable AI engineers know both frameworks—concepts transfer easily

source: ZipRecruiter

Tensors and variables

- TensorFlow operates on multidimensional arrays called **tensors** objects
- On tensors tf implements:
 - Canonical matrix operations
 - ML specialized ops
- Tensors are **immutable**
- To store model weights TensorFlow use a **tf.Variable**

```
[11] 1 import tensorflow as tf
      2
      3 x = tf.constant([[1., 2., 3.],
      4                  [4., 5., 6.]])
      5
      6 print(x)
      7 print(x.shape)
      8 print(x.dtype)
```

```
tf.Tensor(
[[1. 2. 3.]
 [4. 5. 6.]], shape=(2, 3), dtype=float32)
(2, 3)
<dtype: 'float32'>
```

```
[12] 1 x + x
```

```
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[ 2.,  4.,  6.],
       [ 8., 10., 12.]], dtype=float32)>
```

```
[13] 1 tf.nn.softmax(x, axis=-1)
```

```
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[0.09003057, 0.24472848, 0.66524094],
       [0.09003057, 0.24472848, 0.66524094]], dtype=float32)>
```

Automatic differentiation

- Optimization through **gradient descent** is a ML milestone
- To compute gradients tf implements automatic differentiation

```
[15] 1 x = tf.Variable(1.0)
      2
      3 def f(x):
      4     y = x**2 + 2*x - 5
      5     return y
      6
      7 f(x)
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=-2.0>
```

```
[16] 1 with tf.GradientTape() as tape:
      2     y = f(x)
      3
      4 g_x = tape.gradient(y, x) # g(x) = dy/dx
      5
      6 g_x
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=4.0>
```

The [tf.GradientTape](#) API computes the gradient with respect to some inputs

Automatic differentiation

- Gradient of a loss with respect to two variables

```
[3] 1 import tensorflow as tf
    2
    3 w = tf.Variable(tf.random.normal((3, 2)), name='w')
    4 b = tf.Variable(tf.zeros(2, dtype=tf.float32), name='b')
    5 x = [[1., 2., 3.]]
    6
    7 with tf.GradientTape(persistent=True) as tape:
    8     y = x @ w + b
    9     loss = tf.reduce_mean(y**2)
   10
   11 [dl_dw, dl_db] = tape.gradient(loss, [w, b])
   12 print(dl_dw)
   13 print(dl_db)

tf.Tensor(
[[0.07654426 0.67659944]
 [0.15308851 1.3531989 ]
 [0.22963277 2.0297983 ]], shape=(3, 2), dtype=float32)
tf.Tensor([0.07654426 0.67659944], shape=(2,), dtype=float32)
```


Tensorflow today

- In modern TensorFlow, you write code in standard Python (**eager execution**), and when performance is needed, TensorFlow automatically converts it into an optimized computation graph using `tf.function`.
 - Eager execution is the default
 - Operations run immediately
 - Easy debugging (like standard Python)
- From Python to Graph
 - `tf.function` converts Python code into optimized graphs
 - Improves performance and portability
 - Used automatically in many APIs (e.g., `model.fit()`)

Modules, layers, and models

- To do ML in TensorFlow, you are likely to need to define, save, and restore a model, which abstractly is
 - A function that computes something on tensors (a forward pass)
 - Some variables that can be updated in response to training
- Most models are made of layers
- **Layers are functions with a known mathematical structure** that can be reused and have **trainable variables**
- We will use **Keras**, high-level implementations of layers and models, built on the same foundational class [*tf.Module*](#)

- ***tf.keras.layers.Layer*** is the base class of all Keras layers, and it inherits from *tf.Module*
- Also provides a full-featured model class called ***tf.keras.Model***
- In general, Layer class is used to define inner computation blocks, and the Model class to define the outer model
- The Model class has the same API as Layer but exposing also:
 - Built-in training, evaluation, and prediction loops
 - The list of its inner layers, via the `model.layers` property.
 - Saving and serialization APIs
- See [Making new Layers and Models via subclassing](#)

A Complete Machine Learning Pipeline

- **Dataset:** collect, load, and preprocess input data
- **Model:** define the layers and the forward computation
- **Training:** compute the loss and update weights through backpropagation
- **Evaluation:** assess model accuracy and generalization on validation/test data

```
import tensorflow as tf

# 1. Dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize data
x_train = x_train / 255.0
x_test = x_test / 255.0

# 2. Model
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])

# 3. Training
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=['accuracy']
)

model.fit(x_train, y_train, epochs=5)

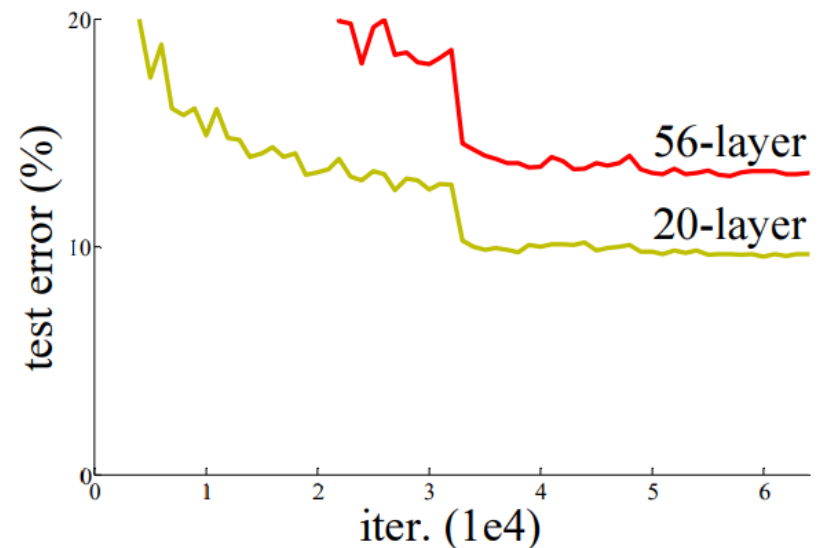
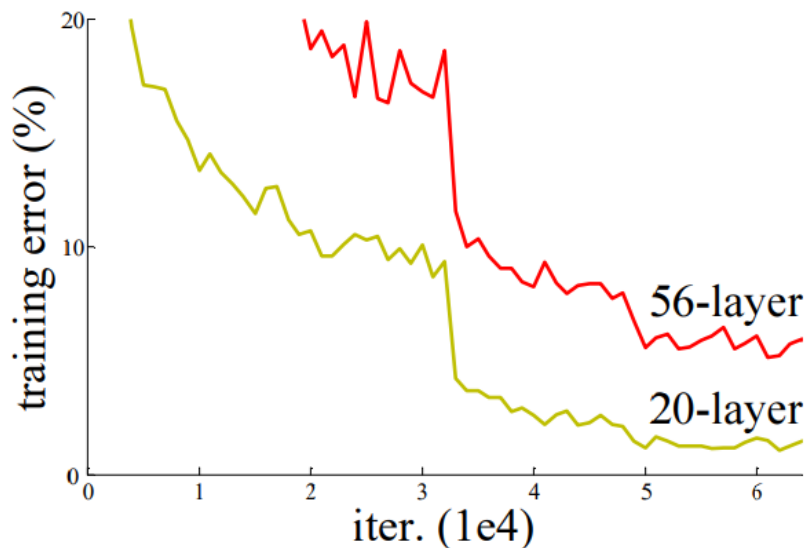
# 4. Evaluation
model.evaluate(x_test, y_test)
```

Assignment

ResNet

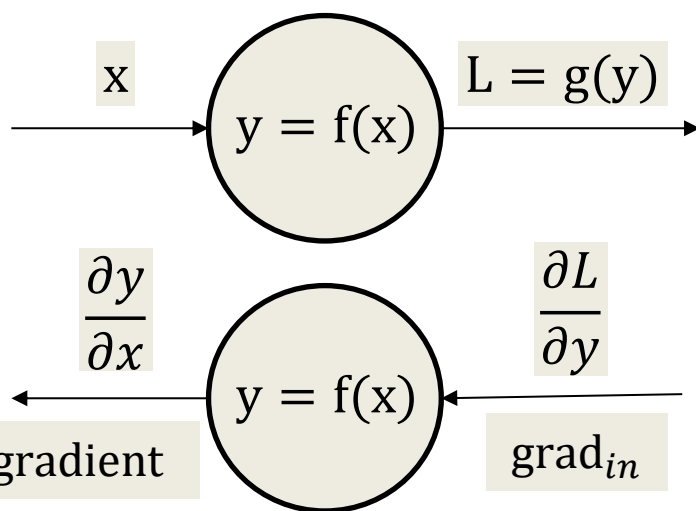
ResNet - Intuition

- Experimentally, with the network **depth increasing**, **accuracy** gets saturated (which might be unsurprising) and then **degrades rapidly**
- Unintuitively more layers = less accuracy
- This is due to two reasons:
 - The vanishing of the gradient
 - The degradation problem



ResNet – Gradient vanishing

- A node in a computational graph can compute two things without even being aware of the rest of the graph :
 - the output of the node (forward pass, top line in the picture below)
 - the local gradient of the node (backward pass, bottom line in the picture below)
- Local gradients of a node are the derivatives of the output of the node with respect to each of the inputs
- Essence of backpropagation: for a **single node**, we find the derivative of the output w.r.t a variable by **multiplying its local gradient with the upstream gradient** that we receive from the upstream step



chain rule

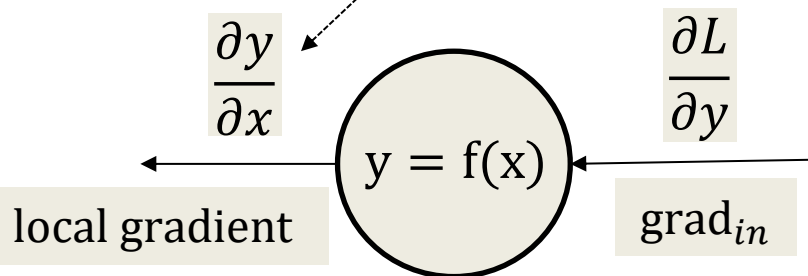
$$\frac{\partial L}{\partial x} = g'(f(x)) \cdot f'(x)$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$$

$$\text{grad}_{out} = \text{grad}_{in} \cdot \text{local gradient}$$

ResNet – Gradient vanishing

- Traditional activation functions (e.g., the hyperbolic tangent) have gradients in $(0,1]$
- **Backpropagation computes gradients by the chain rule**
- **Multiplying** n of these **small numbers** will cause the gradient to decrease exponentially
- As the gradient keeps flowing backward to the initial layers, this value keeps getting multiplied by each local gradient
- The vanishing of the gradient prevents the learning of the early layers



chain rule

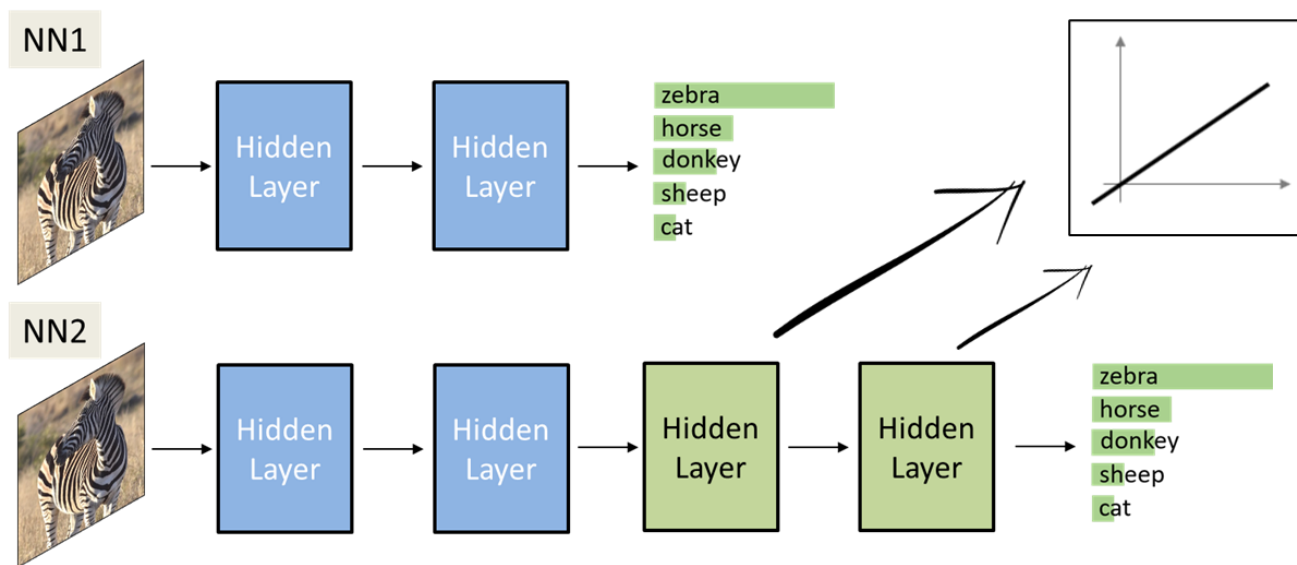
$$\frac{\partial L}{\partial x} = g'(f(x)) \cdot f'(x)$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$$

$$\text{grad}_{out} = \text{grad}_{in} \cdot \text{local gradient}$$

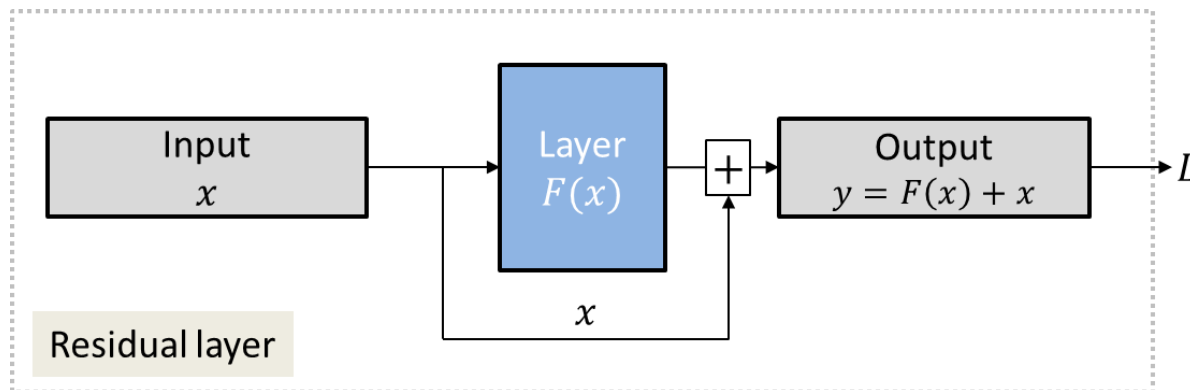
ResNet – Degradation problem

- **Theoretically**, the more layers you add to a neural network, the performance could either go up or stay the same, it should never go down
- Suppose NN1 in its *best possible state* (loss in global minimum)
- If we add more hidden layers (NN2), the new layers should learn the **identity function** $g(x) = x$ to preserve the current best state of the net
- **Learning the identity function is hard** because it requires many parameters to align perfectly to produce no change. It's like passing a message through many people and expecting it to remain exactly unchanged



ResNet – Residual learning

- We introduce the *ResNet block*, a layer with **skip** connections

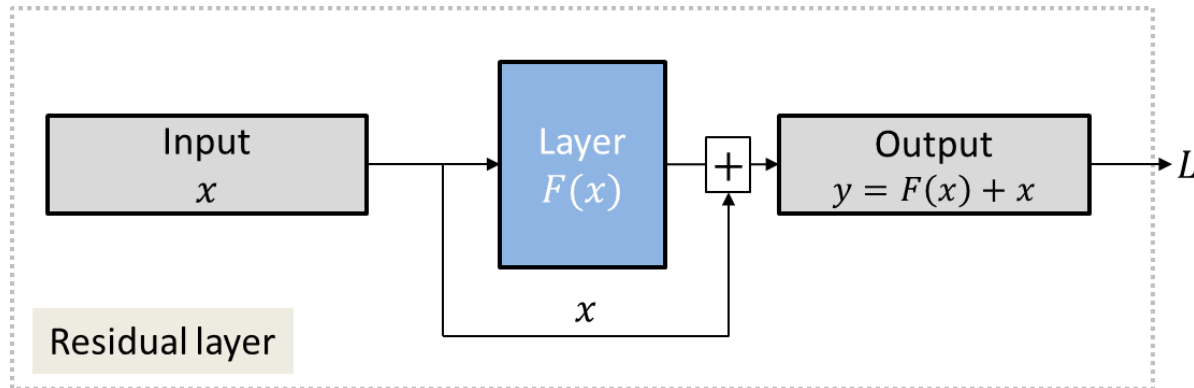


- Defining $y = F(x) + x$, the layer must learn $F(x) = y - x$, which is the **residual** mapping
- In this scenario, learning **the identity function** means learning the **zero** function $y = 0 + x = x$
- Easy to learn, set all weights to zero
- No vanishing gradient: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial L}{\partial y} (F'(x) + 1) = \frac{\partial L}{\partial y} + \frac{\partial L}{\partial y} F'(x)$

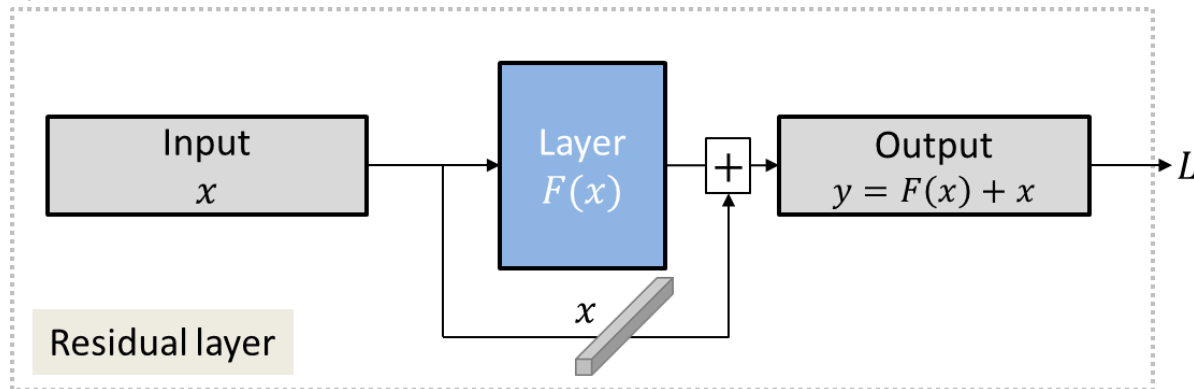
The **error signal (incoming gradient)** is passed through, unmodified by the **local gradient**

ResNet – Residual learning

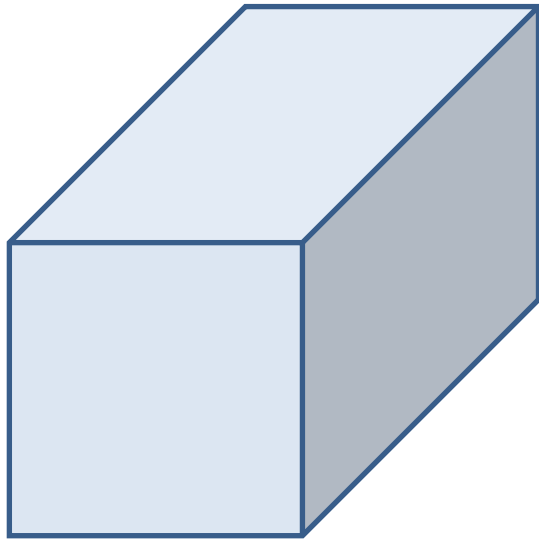
- The architecture depends on the dimension of the output w.r.t. the input



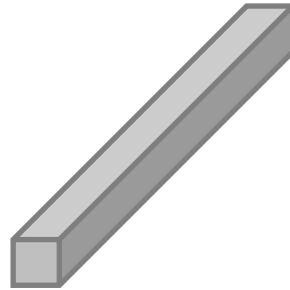
- If the input and output have the same dimensions, the shortcut connection can be implemented as an identity mapping
- Otherwise, a 1×1 convolution is used to match dimensions



ResNet - 1x1 convolution



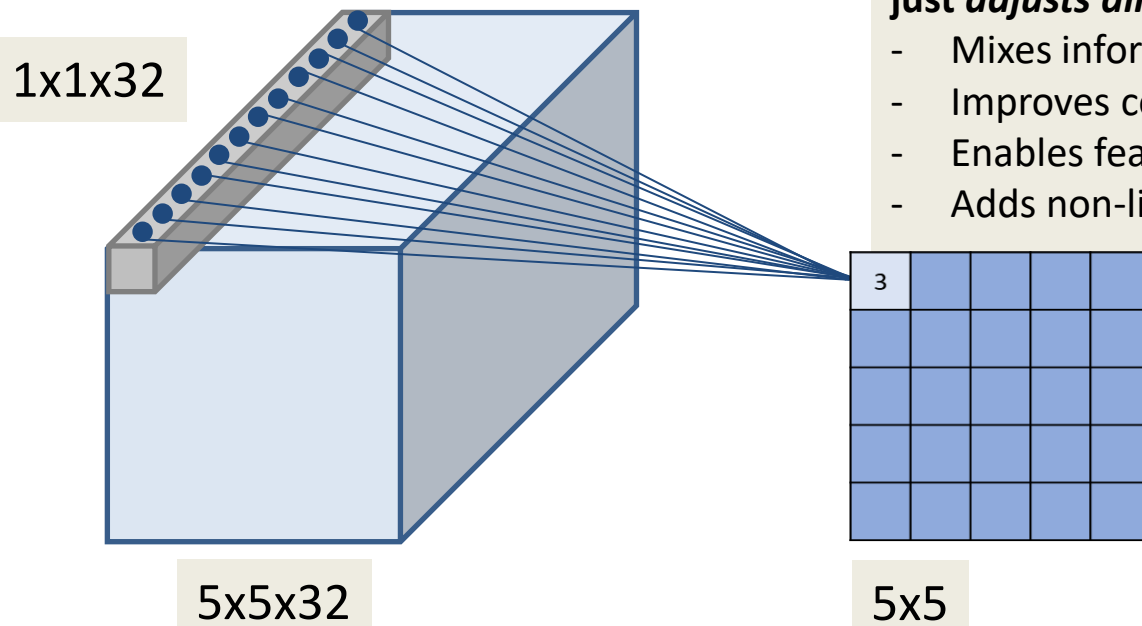
5x5x32



1x1x32

A 1x1 convolution will take an **element-wise product** and then apply a Relu non-linearity

ResNet - 1x1 convolution



Transforms and mixes features efficiently, not just *adjusts dimensions*

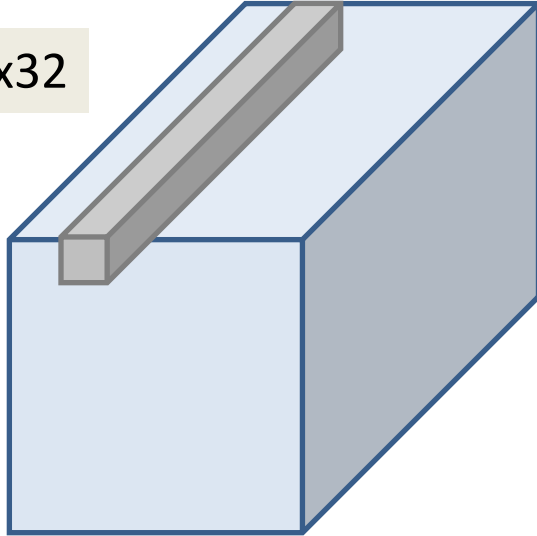
- Mixes information across channels
- Improves computational efficiency
- Enables feature transformation
- Adds non-linearity (with activation)

Can be see a neuron having as input 32 number multiplied by weights → Network In Network [1]

[1] Lin, Min, Qiang Chen, and Shuicheng Yan. "Network In Network." arXiv preprint arXiv:1312.4400 (2013).

ResNet - 1x1 convolution

1x1x32



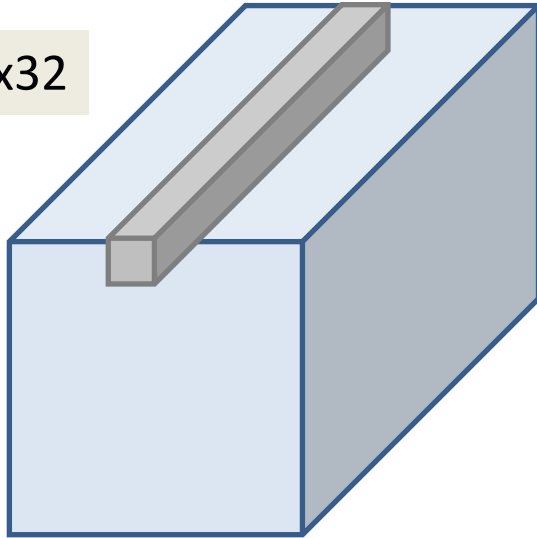
5x5x32

3	5			

5x5

ResNet - 1x1 convolution

1x1x32



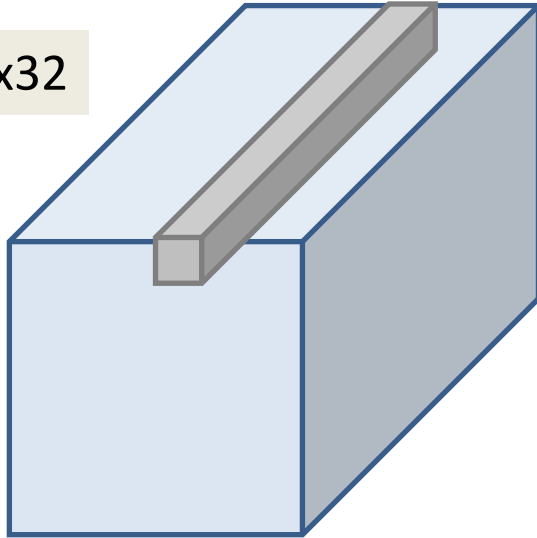
5x5x32

3	5	2		

5x5

ResNet - 1x1 convolution

1x1x32

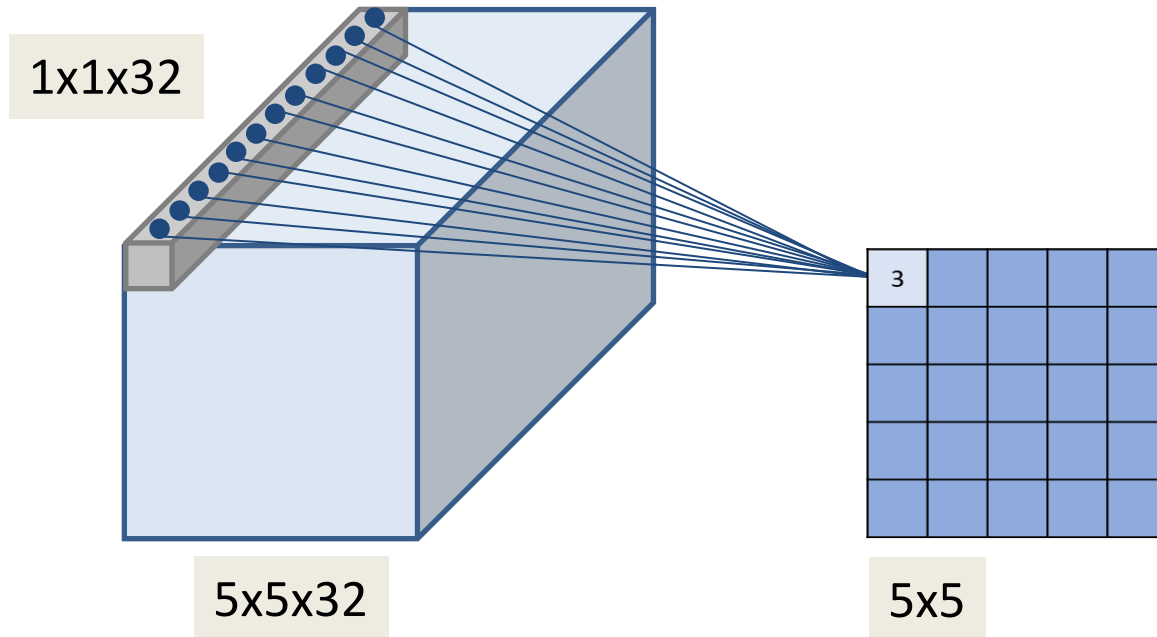


5x5x32

3	5	2	7	

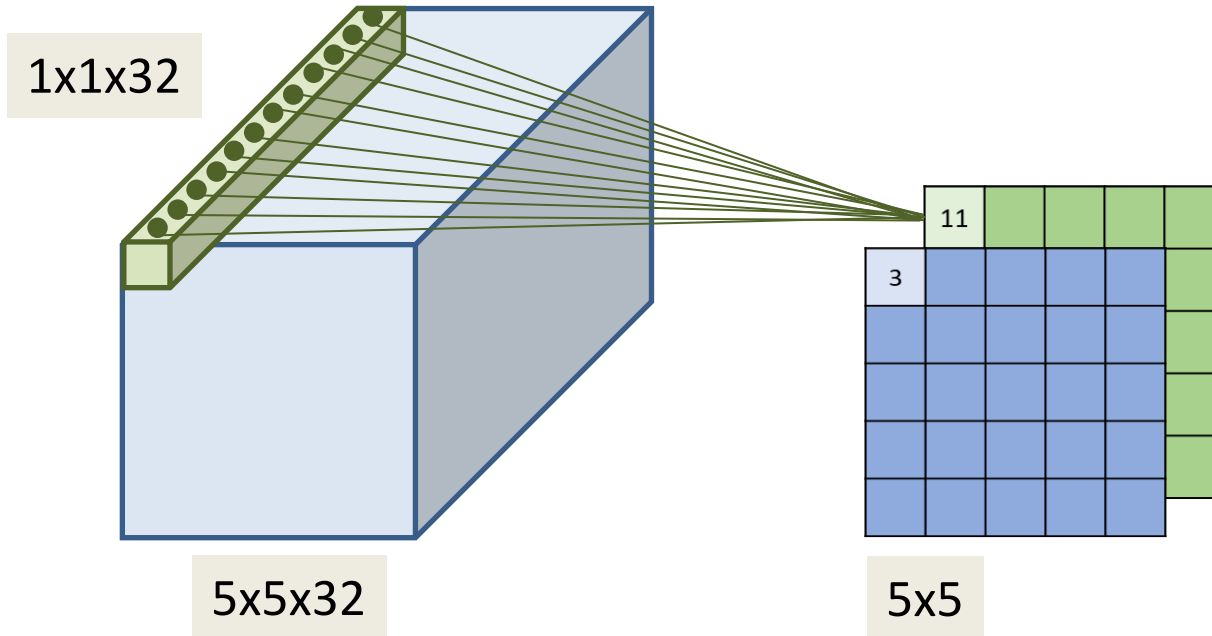
5x5

ResNet - 1x1 convolution



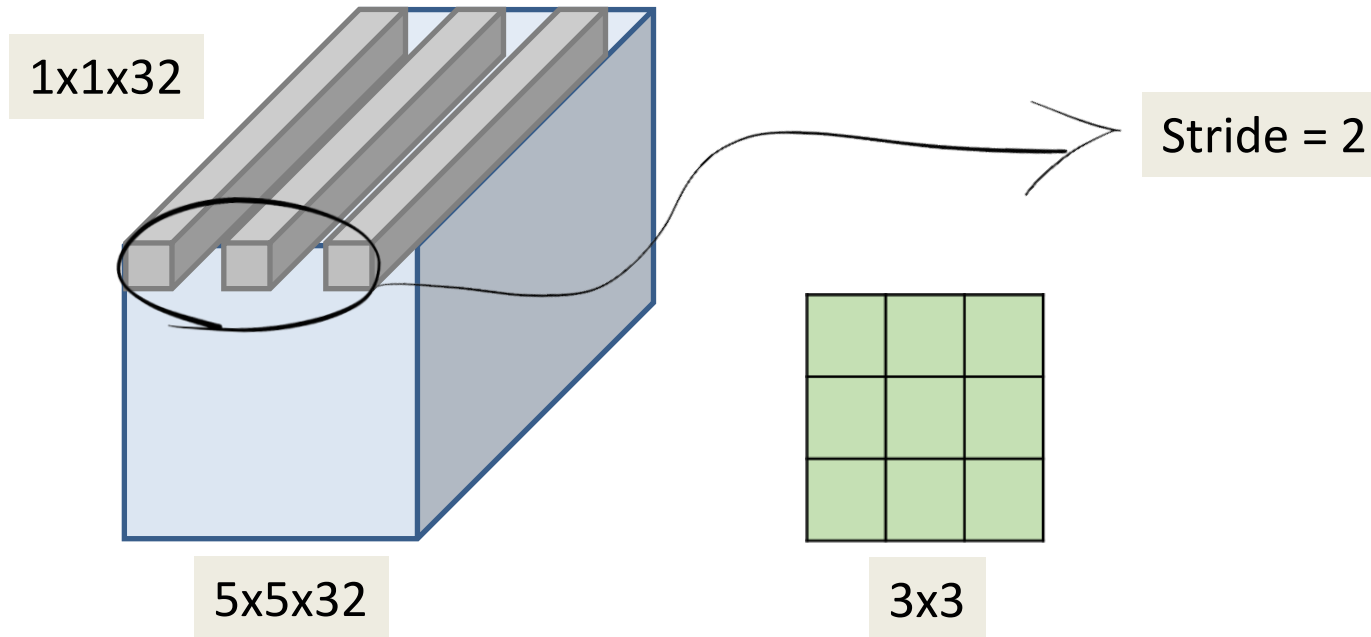
Using N different 1×1 convolutional filters we obtain volumes of different depth

ResNet - 1x1 convolution



Using N different 1x1 convolutional filters we obtain volumes of different depth

ResNet - 1x1 convolution



Concerning ResNet, since there's no pooling layer within the residual block, the dimension is downscaled by 1×1 convolution with strides 2